

Estrutura de Dados (CC4652)

Aula 4 - Complexidade de Algoritmos (Laboratório)

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2024

Complexidade de Algoritmos

Análise de Algoritmos

1. Suponha que cada expressão abaixo represente o tempo $T(n)$ consumido por um algoritmo para resolver um problema de tamanho n . Escreva os termo(s) dominante(s) para valores muito grandes de n e especifique o limite assintótico superior $O(n)$ possível para cada algoritmo.

Expressão	Termo Dominante	Notação Big-O
$10 + 0.001n^3 + 0.025n$	$0.001 \cdot n^{**3}$	$f(n) = O(0.001 \cdot n^{**3})$
$500n + 100n^{1.5} + 10\log_{10}(n)$	$100 \cdot n^{**1.5}$	$f(n) = O(100 \cdot n^{**1.5})$
$1000n + 0.03n^2$	$0.03 \cdot n^{**2}$	$f(n) = O(0.03 \cdot n^{**2})$
$n^2 \log_2(n) + n(\log_2(n))^2$	$n^2 \log_2(n)$	$f(n) = O(n^2 \log_2(n))$
$7 + \log_2(n)$	$\log_2(n)$	$f(n) = O(\log_2(n))$
$2^n + 10$	2^{**n}	$O(2^{**n})$

Complexidade de Algoritmos

Análise de Algoritmos

2. Analise o código ao lado e encontre sua complexidade. Utilize a notação Big-O

$$f(n) = 1 + 1 + 1 + 1 + (\log(n)) + n$$

$$f(n) = 4 + (\log(n)) + n$$

$$f(n) = O(n)$$

```
1  #include <stdio.h>
2
3  int main(){
4      1 int n;
5      1 scanf("%d", &n);
6      1 int i = 1;
7      1 int j = 1;
8
9      while(i <= n) {
10         log2(n) i = i * 2;
11     }
12
13     while(j <= n) {
14         n j = j + 1;
15     }
16 }
```

Complexidade de Algoritmos

Análise de Algoritmos

3. Analise o código ao lado e encontre sua complexidade. Utilize a notação Big-O

$$f(n)=1+1+1+1+1+(n*n*1)+1+(n*1*1)$$

$$f(n)=5+(n**2)+n$$

$$f(n)=O(n**2)$$

```
1  #include <stdio.h>
2
3  int main(){
4      1  int n;
5      1  scanf("%d", &n);
6      1  int i = 1;
7      1  int j = 1;
8      1  int cont = 0;
9
10     n  for(int i = 1; i <= n; i = i + 1) {
11         n  for(int j = 1; j <= n; j = j + 1) {
12             } 1 cont++;
13         }
14     1  int k = 1;
15
16     n  while(k <= n) {
17         1  k++;
18         1  cont++;
19     }
20 }
```

Complexidade de Algoritmos

Análise de Algoritmos

4. Analise o código ao lado e encontre sua complexidade. Utilize a notação Big-O

$$f(n) = 1 + 1 + 1 + (n * n * n * 1)$$

$$f(n) = 3 + n^{**3}$$

$$f(n) = O(n^{**3})$$

```
1  #include <stdio.h>
2
3  int main(){
4      1  int n;
5      1  scanf("%d", &n);
6      1  int cont = 0;
7
8      n  for(int i = 1; i <= n; i = i + 1) {
9          n  for(int j = 1; j <= n; j = j + 1) {
10             n  for(int k = 1; k <= n; k = k + 1) {
11                 1  cont++;
12             }
13         }
14     }
15
16 }
```

Complexidade de Algoritmos

Análise de Algoritmos

5. Analise o algoritmo ao lado, que recebe dois vetores, a e b , de tamanhos iguais a n

- ▶ Determine o limite assintótico inferior para o melhor caso em função do parâmetro n
- ▶ O limite assintótico superior para o pior caso em função do parâmetro n
 $f(n) = 2 + (n * 1 * 1 * (\log_3(n) * 1)) = O(n * \log_3(n))$
- ▶ As condições que o vetor a deve satisfazer para caracterizar o melhor caso $5 \leq a[i] \leq 10$

```
1 float f(float* a, float* b, int n) {
2     1 int i, j;
3     1 float s = 0.0;
4     1 for (i=1; i<n; i++) { n
5         if (a[i]>10) { 1
6             n for (j=n-1; j>=0; j--)
7                 s += a[i]+b[j];
8         }
9         else if (a[i]<5) {1
10            (n**2-n)/2 for (j=n; j<n*n; j+=2)
11                s += a[i]+b[j]; 1
12        }
13        else
14        {
15            log3(n) for (j=1; j<n; j=3*j)
16                s += a[i]+b[j];
17        }
18    }
19    return s;
```

Complexidade de Algoritmos

Análise de Algoritmos

6. Desenvolva um algoritmo para quebrar uma senha composta por 5 dígitos (com abordagem força-bruta). Considere que a senha é puramente numérica. Faça a análise desse algoritmo. Qual é a ordem de grandeza da complexidade (utilize a notação Big-O)? O algoritmo seria eficiente para quebrar a senha se ela tivesse 10 dígitos? Justifique sua resposta com a análise de complexidade desenvolvida.

$f(n)=O(n^6)$, o algoritmo não seria útil para quebrar uma senha de 10 dígitos pois na lógica desenvolvida é feito um for para cada item da lista e um for para percorrer o todo, assim esse código só descobre se a senha tiver 5 dígitos

Estrutura de Dados (CC4652)

Aula 4 - Complexidade de Algoritmos (Laboratório)

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2024